

Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
geya@LenovoL480: ~/OS/Pra × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i;

    // malloc(): allocate memory for 5 integers
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (i = 0; i < 5; i++)
    {
        a[i] = (i + 1) * 10;
    }

    printf("Memory allocated using malloc:\n");
    for (i = 0; i < 5; i++)
    {
        printf("%d ", a[i]);
    }
}
```

```
}
printf("\n");

// calloc(): allocate memory for 5 integers
int *b = (int *)calloc(5, sizeof(int));

if (b == NULL)
{
    printf("calloc failed\n");
    free(a);
    return 1;
}

printf("\nMemory allocated using calloc:\n");
for (i = 0; i < 5; i++)
{
    printf("%d ", b[i]);
}
printf("\n");

// realloc(): increase size of malloc memory
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL)
{
```

```

        printf("realloc failed\n");
        free(b);
        return 1;
    }

    for (i = 5; i < 10; i++)
    {
        a[i] = (i + 1) * 10;
    }

    printf("\nMemory after realloc:\n");
    for (i = 0; i < 10; i++)
    {
        printf("%d ", a[i]);
    }
    printf("\n");

    // free(): release allocated memory
    free(a);
    free(b);

    // Deliberate memory leak
    int *leak = (int *)malloc(100 * sizeof(int));

    if (leak != NULL)
    {
        leak[0] = 100;
        printf("\nDeliberate memory leak created.\n");
    }

    return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Practical/Week 8$ ./task1
```

```
Memory allocated using malloc:
```

```
10 20 30 40 50
```

```
Memory allocated using calloc:
```

```
0 0 0 0 0
```

```
Memory after realloc:
```

```
10 20 30 40 50 60 70 80 90 100
```

```
Deliberate memory leak created.
```

```
geya@LenovoL480:~/OS/Practical/Week 8$ |
```